
DAMNED: Digital Analog Modular NeoPixel Enable Display

- an extensible IoT platform

1. Features

- ESP32-S3 core with 4MB Flash, 2MB PSRAM
- Support for Adafruit CircuitPython and associated libraries
- I2C, I2S, SPI, UART connectivity
- Universal USB-C power connector
- 24 NeoPixel RGB LED ring display
- Continuously rotatable stepper motor arm
- WiFi connectivity
- LiPo battery support
- Real-time clock

2. Applications

- Environmental monitoring systems
- API interface devices
- Minimal games
- Wayfinding devices
- Musical instruments
- Time management systems
- Security alert and monitoring systems
- Training and quiz aides

3. Description

The DAMNED (Digital Analog Modular NeoPixel Enabled Display) is an extensible IoT platform. Built on an ESP32-S3 core, it can be programmed using CircuitPython. Multiple digital communication interfaces exist to allow for a wide range of compatible devices to augment the capabilities of the core DAMNED system. WiFi and a system RTC are available for real-time, modern IoT applications that allow for interfacing with API's. Analog input is also an option with several channels accepting signals from analog devices.

Core display output for data visualization includes a ring of 24 individually addressable RGB LEDs. These programmable LEDs allow for any light display from the most basic single color ring to complex configurations of moving patterns. A continuously rotatable stepper motor arm is also available with rotational speeds on the order of 60 RPM.

The device is housed in an enclosure that allows integrated access to all communications ports and power input. A magnetic faceplate allows for easy disassembly and the addition of a battery or replacement parts.

4. Inputs and outputs

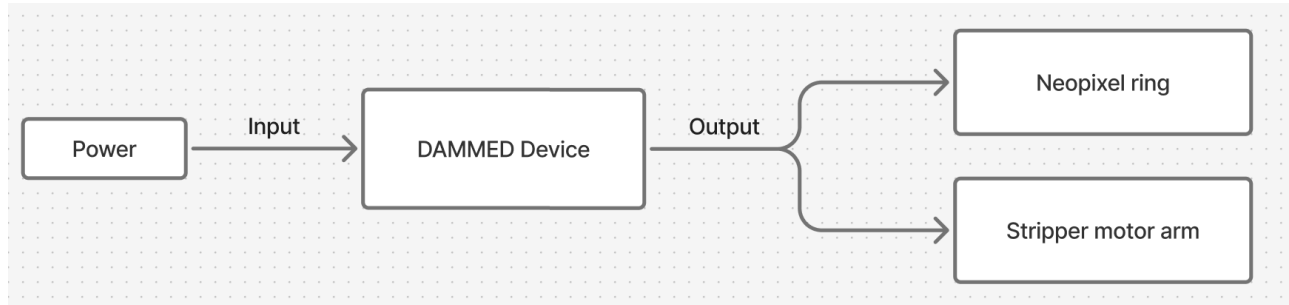


Figure 1: input and output diagram of the DAMMED Device

5. Absolute maximum ratings

	Max	Unit
USB supply voltage	6.5	V
LDO regulator input voltage	12	V
LDO regulator output current	1.5	A

Table 1: absolute maximum ratings

6. Electrical Characteristics

	Min	Max	Unit
USB supply voltage	4.75	5.25	V
LDO regulator input voltage	2.7	10	V
Operating temperature	0	37.8	°C

Table 2: Recommended Operating Conditions

		Unit
Output voltage	4	V

Table 3: Nominal stepper motor supply output voltage with 5 V input and 1k load

6.1 Measured Characteristics

Vin (V)	Vout (V)	Iin (mA)	Iout (mA)	Pin (W)	Pout (W)	Peff (%)
4.98	3.98	304	254	1.514	1.01	66,7

Table 4: Stepper motor supply efficiency with 5 V input and 10 ohm load

Vin (V)	Vout (V)	In (mA)	Percent difference of Vout from 4 V
6	3.93	456	1.75
5.8	3.93	355	1.75
5.6	3.93	454	1.75
5.4	3.93	453	1.75
5.2	3.93	452	1.75
5	3.93	451	1.75
4.8	3.92	450	2
4.6	3.91	450	2.25
4.4	3.78	436	5.5
4.2	3.58	413	10.5

Table 5: Stepper motor supply input voltage range under 10 ohm load

Vin (V)	Vout (V)	Percent difference of Vout from 4 V
5	3.93	1.75

Table 6: Dropout voltage required to maintain motor supply (4 V) to within 2% under a 10-ohm load

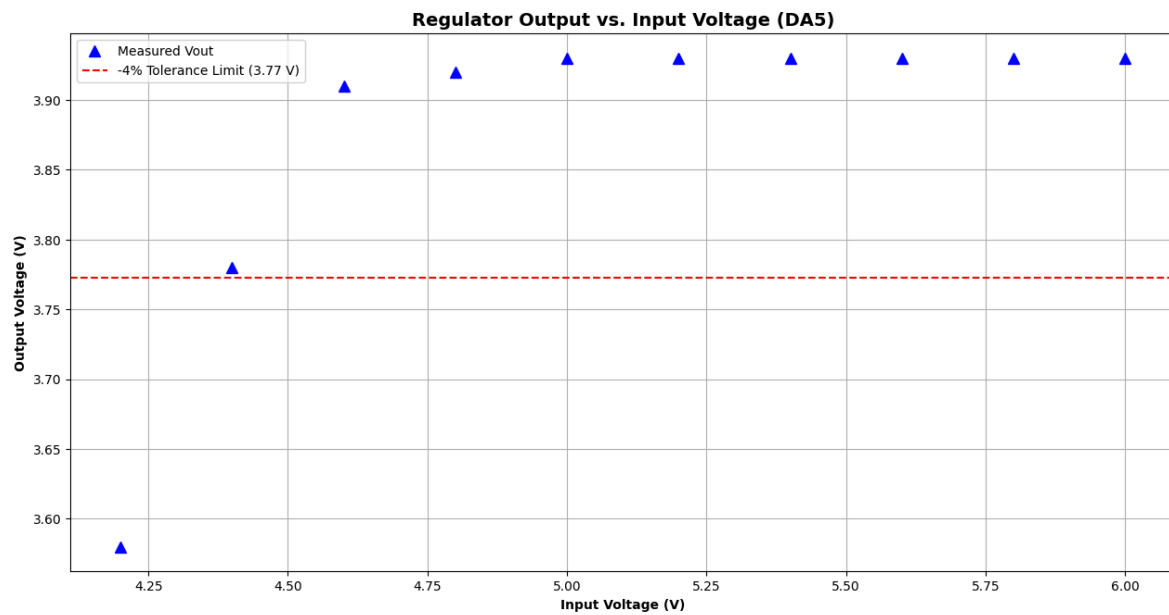


Figure 2: Vin vs Vout for stepper motor voltage regulator

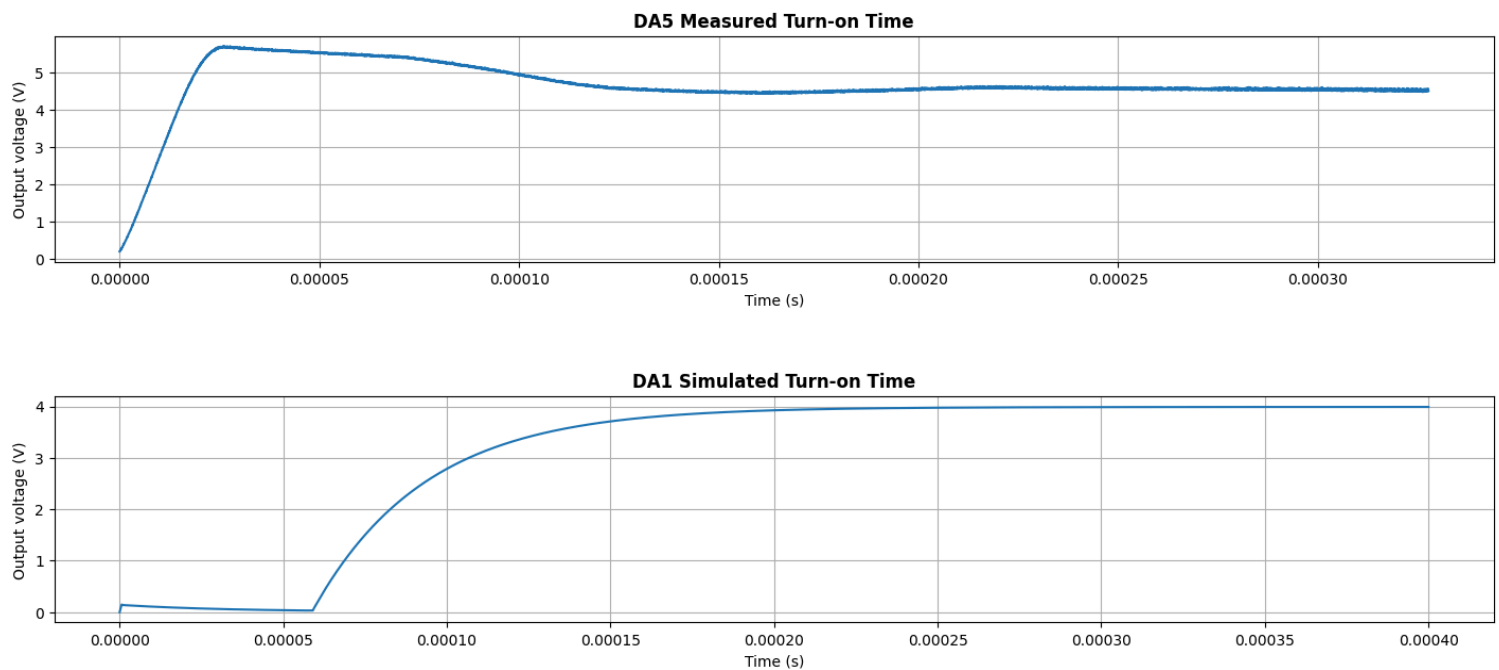
Comparison of Regulator Turn-on Times

Figure 3: voltage regulator turn on, measured, and simulated

The turn-on time of the stepper motor voltage regulator is around 200 μ s.

6.2 Modeled Characteristics

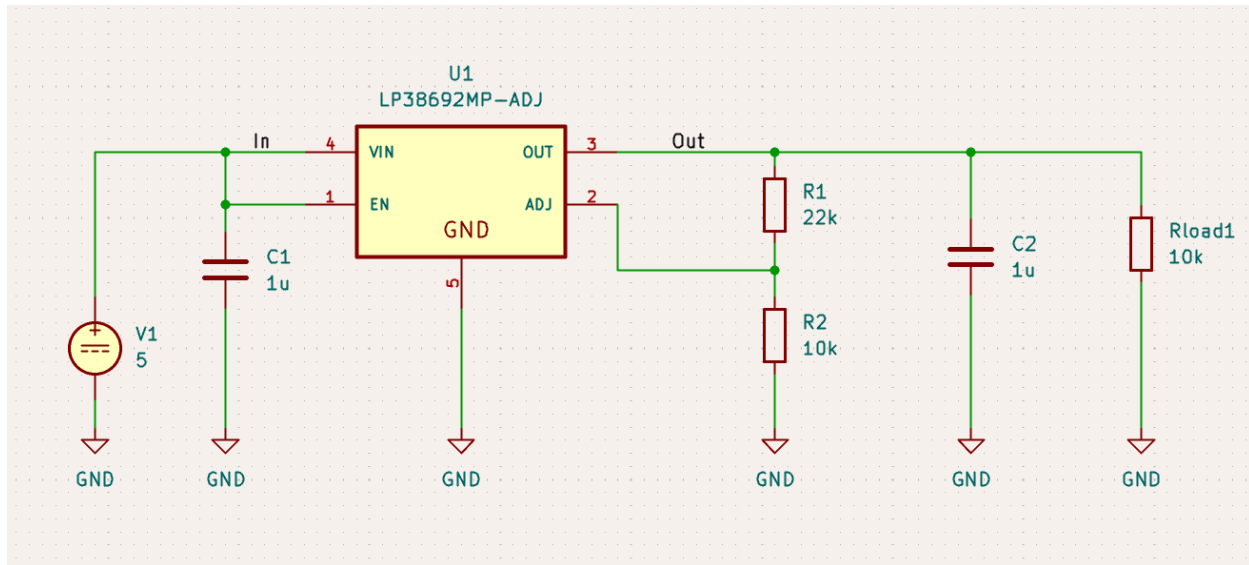


Figure 4: Stepper Motor Voltage Regulator Circuit

Line Regulation of the Motor Control System

	Test conditions	Typical	Unit
$\Delta V_{OUT}/\Delta V_{IN}$ Output voltage line regulation	$V_{out} = 5\text{ V}$ $I_{out} = 1\text{ A}$	0.028	%V

Vin (V)	Vout (V)
1	0.00178
2	1.55
3	2.55
4	3.55
5	4
6	4
7	4.01
8	4.01
9	4.01
10	4.01

Table 7: Input voltage vs output voltage at 1k load, motor control system

Rload	Vin (V)	Iin (A)	Vout (V)	Iout (A)	Peff (%)
1	5	1.5	1.5	1.5	30
2	5	1.5	3	1.5	60
3	5	1.33	3.99	1.33	79.8
20	5	0.2	4	0.2	80
30	5	0.133	4	0.133	80
1k	5	4.13×10^{-3}	4	4×10^{-3}	77.5
10k	5	5.26×10^{-4}	4	4×10^{-4}	60.8
100k	5	1.66×10^{-4}	4	4×10^{-5}	19.3
1M	5	1.3×10^{-4}	4	4×10^{-6}	1.9

Table 8: Load Regulation and efficiency of motor control system

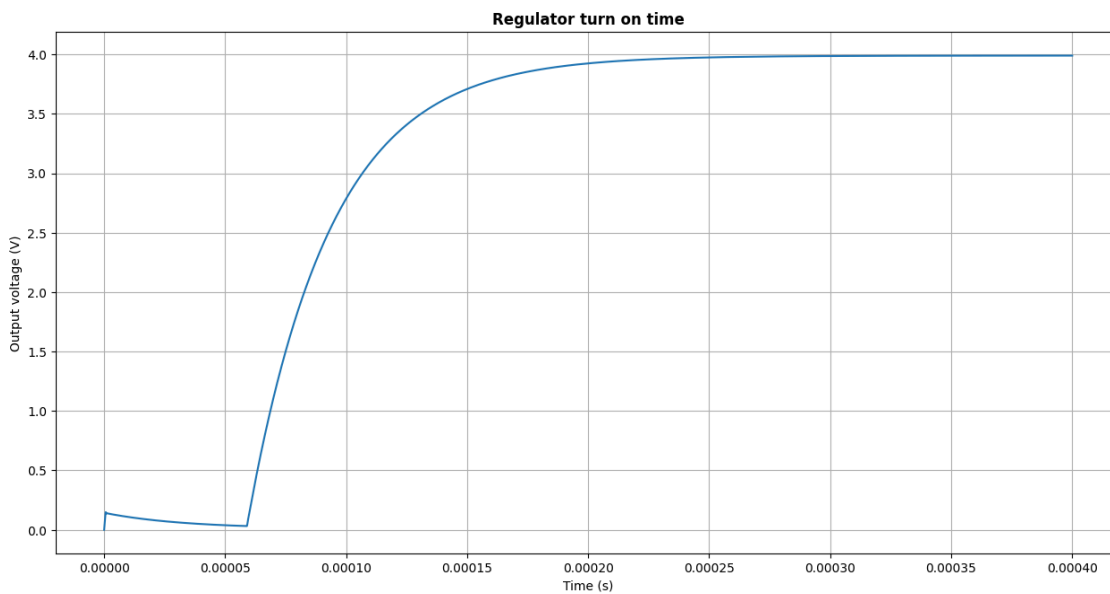
Turn on the time of the motor control system

Figure 5: voltage regulator turns on, modeled

Typical turn-on time for the motor control system power supply is about $200 \mu\text{s}$.

7. Detailed Description

7.1 Functional Block Diagram

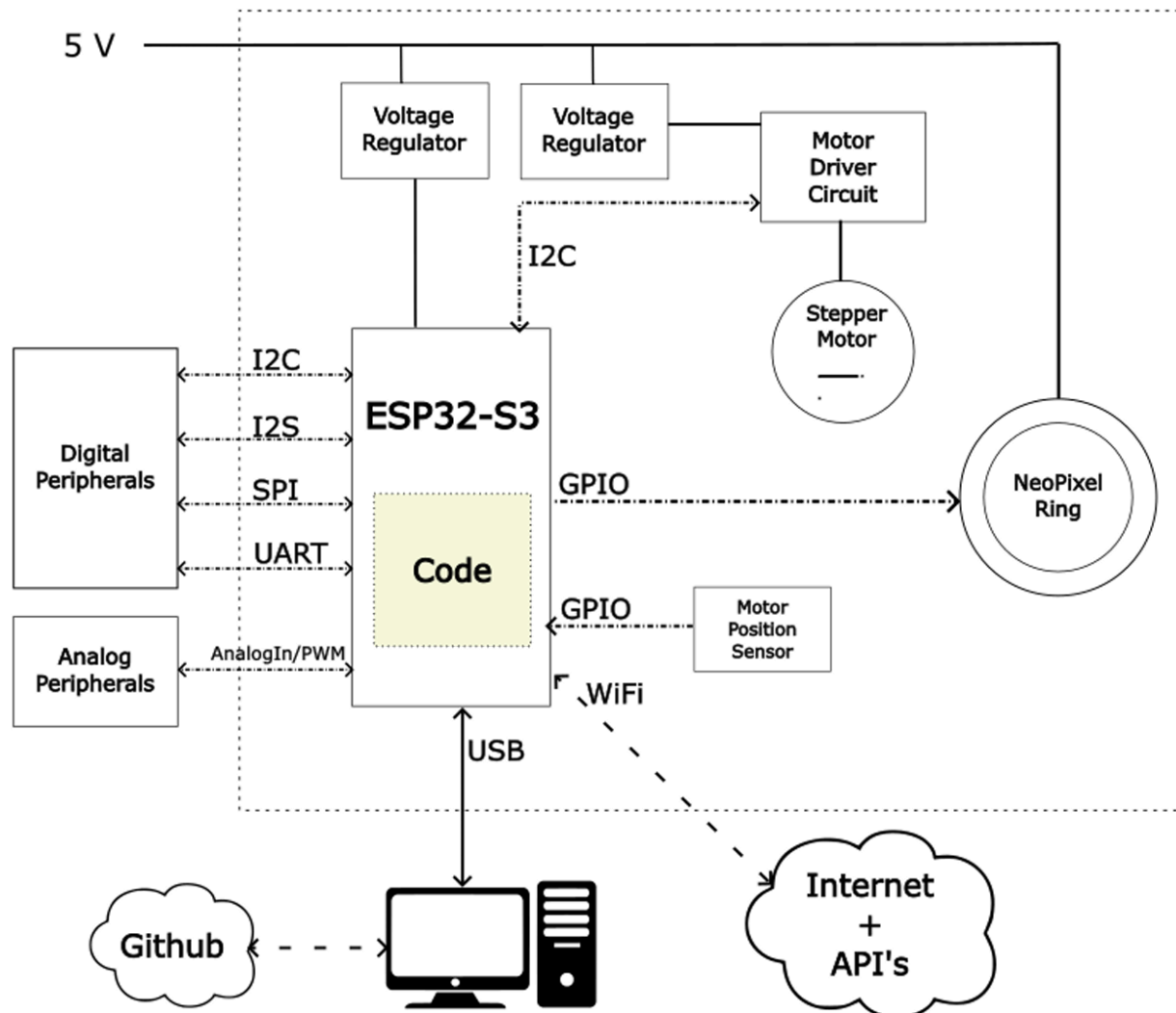


Figure 6: Complete DAMNED Project Block Diagram (Matt Lamparter 2025)

The fundamental operation of the DAMNED begins when power is supplied to the ESP32-S3. The microcontroller immediately triggers a homing sequence via an I2C connection, utilizing a Hall sensor connected to a GPIO pin to accurately locate the stepper motor's starting position. After the arm returns to this home position, the NeoPixel ring defaults to a continuous, looping "snake" light pattern.

The system's block diagram demonstrates the flow of power and data from input to output. Driven by a 5V USB-C power source, the current routes through a voltage regulator to operate the ESP32-S3. This power is also distributed to the Motor Drive Circuit to enable physical movement and to the NeoPixel Ring for visual display.

7.2 Feature Description

7.2.1 Power System

In the DAMNED, the ESP32-S3 stores the code and sends functions to the corresponding components. The system is powered via a USB-C connection that provides 5V input, which is suitable for standard USB power banks for standalone operation. This 5V input passes through a voltage regulator and an internal power management unit to convert it into an operating voltage of 3.3V, which powers the ESP32-S3 microcontroller, the NeoPixel Ring, and the Motor Drive Circuit [1].

7.2.2 Sensors

When starting up the DAMNED, the ESP32-S3 signals a homing process where an integrated Hall Sensor (connected to board D16) searches for the magnetic field associated with the magnet inside the stepper motor arm. The motor arm rotates until it triggers the detection from the Hall sensor and establishes a 90-degree zero point.

- **Inertial Measurement Unit (IMU):** The system utilizes an Adafruit ICM20948 9-DoF IMU connected via the STEMMA QT I2C chain (Notice: when plugging in the STEMMA QT ports, because there are 3 ports, we have to identify the address of each port. For my project, I used port 0x69). The IMU reads real-time gravitational acceleration vectors (m/s^2) on the X and Y axes, allowing the microcontroller to calculate the absolute roll and pitch of the device using arc-tangent mathematics.

7.2.3 Software and Processor

- **Microcontroller:** ESP32-S3 chip featuring 4MB Flash and 2MB PSRAM [1].
- **Language:** Adafruit CircuitPython.
- **Concurrency:** The system utilizes the asyncio library (and dependent `adafruit_ticks` module) to perform asynchronous, non-blocking hardware control [5]. This allows a high-frequency (100Hz) hardware task for reading the IMU and steering the motor to run concurrently with a lower-frequency (2Hz) game engine task that calculates Cartesian object mapping, without either process blocking the other.
- **Library Dependencies:** `adafruit_icm20x` [3], `adafruit_character_lcd` [4], `motorFunctions.py`, `neoPixelFunctions.py`

7.2.4 Web connectivity

The DAMNED features 2.4 GHz WiFi connectivity via an embedded ESP32-S3 microcontroller. The module supports 802.11 b/g/n network connectivity.

Device MAC address
10-20-BA-09-37-D0

7.2.5 Display

7.2.5.1 Lights

The device utilizes a circular ring of 24 individually addressable RGB NeoPixels connected to the [board.D5](#). In this application, it acts as a spatial radar array. The custom software engine maps virtual 2D coordinates to specific LEDs on the ring, modulating color to indicate object type (Yellow for Coins, Blue for Icebergs) and brightness to indicate proximity.

7.2.5.2 Motor arm

The I2C-controlled stepper motor arm acts as a dynamic physical compass [2]. It maps the user's physical tilt of the device to continuous proportional rotational velocity, establishing the player's vector heading within the virtual space.

	AO1	AO2	BO1	BO2
h2A	0	4	0	0
h32	0	0	4	0
h2E	4	0	0	0
h36	0	0	0	4

Table 8: Stepper motor register to terminal mapping

7.2.5.3 Auxiliary display

A 16x2 Character LCD Backpack is connected via the STEMMA QT I2C chain (Address 0x20) [4]. It provides real-time telemetry, displaying the system state, calibration instructions, survival time, and score.

8. Application and Implementation

8.1 Applications

The DAMNED functions can be used as a high-precision environmental logger by interfacing with digital sensors via the I2C bus. It can be programmed to read temperature data at fixed intervals (e.g., once per second) and append that data to a local .csv file or transmit it to the cloud for remote monitoring.

- Sensors Used: AHT20 (Aosong/ASAIR) and MCP9808 (Microchip Technology).
- AHT20 Metrics: Based on experimental data, the AHT20 shows a Decay Time of approximately 130s, a high stability with low COV, and minimal Drift over time.
- MCP9808 Metrics: This sensor provides higher industrial accuracy with a Decay Time of approximately 155s, though it exhibits slightly higher COV (noise) in baseline readings compared to the AHT20.

Automated Time and Temperature Display

The DAMNED can also be used as an "Internet of Things" (IoT) clock and gauge.

- Time Synchronization: The device connects to WiFi and uses the NTP (Network Time Protocol) to fetch the current UTC, which it then uses to set the internal Real Time Clock (RTC).
- Mapping Time to NeoPixel Ring: The 12-LED NeoPixel ring displays time by mapping the 60-minute cycle and 12-hour cycle to LED positions. A yellow dot represents the current minute, and a blue dot represents the current hour. If the hour and minute overlap, the LED turns white.
- Mapping Temperature to Stepper Arm: The device fetches local hourly weather via an API call. The temperature range of 0F to 100F is mapped linearly to the stepper motor: 0F is at the 8 o'clock position, and 100F is at the 4 o'clock position.

Here is what it would look like:

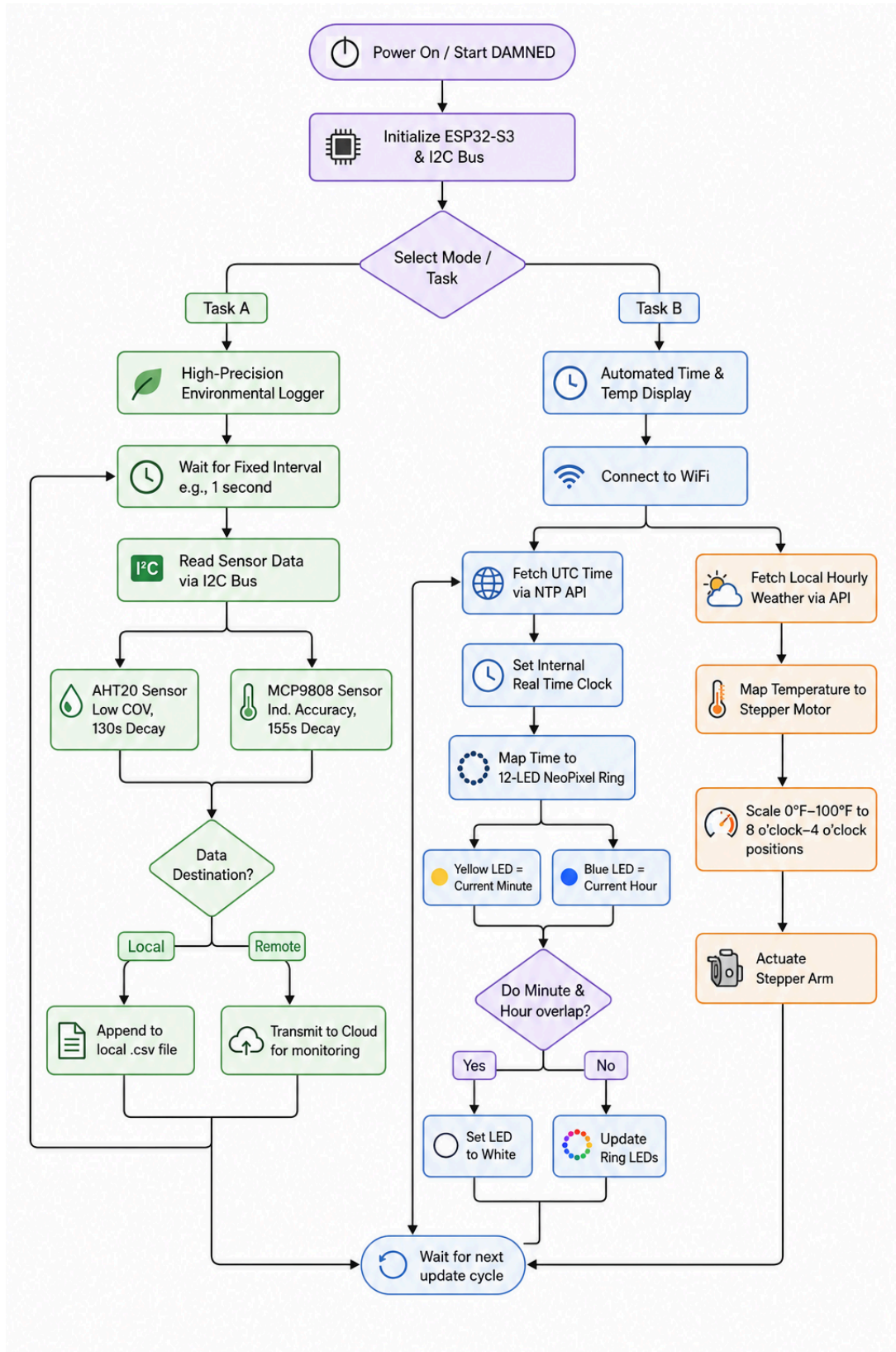


Figure 7: Flowchart detailing the use cases of the DAMMED device

For my final project, the DAMNED platform is configured as a fully self-contained, physical-computing arcade console. Rather than acting as a traditional environmental logger, the device leverages its physical hardware as both the primary input controller and the dynamic visual output array for a 2D spatial survival game.

The primary sensor is an **Adafruit ICM20948 9-DoF Inertial Measurement Unit (IMU)** connected via the STEMMA QT I2C bus (Address `0x69`). Additionally, an integrated **Hall effect sensor (board D16)** is utilized during the boot sequence to detect the internal magnet of the motor arm to establish the start position of the motor arm

The system displays real-time interactive game states rather than static telemetry. This includes the player's vector heading, a spatial radar indicating approaching virtual objects (Coins to collect and Icebergs to dodge), object proximity, and game telemetry (survival time and score), which is outputted to a 16x2 Character LCD Backpack.

The flow of information from the sensors to the hardware displays relies on a custom asynchronous (`asyncio`) game engine:

- **Mapping to the Stepper Motor Arm (Heading):** The ESP32-S3 continuously polls the IMU for real-time gravitational acceleration vectors on the X and Y axes. Using arc-tangent mathematics, the system calculates the absolute tilt of the board. Applying a 10-degree deadzone, this physical tilt angle is mathematically mapped to a proportional rotational velocity. Tilting the board actuates the stepper motor, turning the physical arm to represent the submarine's continuous vector heading.
- **Mapping to the NeoPixel Ring (Spatial Radar):** The software generates a virtual 2D Cartesian plane. As the motor arm establishes a heading, the engine shifts the X and Y coordinates of generated virtual objects. The relative angle of an object is mapped to a specific LED index (0–23) on the NeoPixel ring. The object type is mapped to RGB color values (Yellow for Coins, Blue for Icebergs). Finally, the calculated Cartesian distance to the object is mapped to LED brightness, with distant objects appearing dim and objects at collision distance displaying at maximum brightness. If a collision is registered, the entire ring is mapped to flash Red to indicate a Game Over state.

8.2 Setup

The DAMNED utilizes the ESP32-S3 microcontroller to allow the device to communicate and retrieve data from the ICM20948 Inertial Measurement Unit (IMU) via I2C, along with activating the NeoPixel ring, stepper motor, and character LCD. Assuming that the user has downloaded the `code.py` and `submarine_engine.py` files and attached the correct libraries (including the `asyncio` library and `adafruit_ticks.mpy`) into the `lib` folder on the DAMNED, the device is ready for standalone operation. Upon receiving power, the device will run the `code.py` file and begin a stepper motor homing process using a Hall effect sensor built into the DAMNED (board.D16) to find its absolute 90-degree mechanical zero. Afterwards, the LCD Display and NeoPixel ring will be initialized as outputs, and the device will run a mandatory 2-second environmental gravity calibration by taking 50 rapid samples from the IMU to establish a level "zero" point for the table.

Within the ESP32 of the DAMNED, there lies an asynchronous I/O (`asyncio`) framework that allows for concurrent, non-blocking execution of multiple hardware tasks. By utilizing this asynchronous library, the

ESP32-S3 can read the IMU data and update the stepper motor's proportional velocity at a high frequency (100Hz) without being interrupted by the game's logic. Simultaneously, the `submarine_engine.py` library acts as a local server, calculating a 2D Cartesian plane to continuously track virtual game objects (Coins and Icebergs). By applying mathematical geometry within the code (such as arc-tangent formulas and Cartesian distance equations), it allows the ESP32 to translate physical tilt data into a dynamic vector heading, and translate virtual coordinates into physical visual outputs displayed on the NeoPixel ring radar.

Game State Example: Virtual Cartesian coordinates mathematically mapped to physical 0-23 LED indices on the NeoPixel ring and distances mapped to RGB brightness values.

Additional sensors and peripherals utilized for the DAMNED are connected based on either I2C or GPIO pins. As such, I utilized an external Adafruit ICM20948 9-DoF IMU and a 16x2 Character LCD Backpack connected through the ESP32-S3 STEMMA QT port using SDA and SCL lines. With this connection, the IMU (Address 0x69) sends real-time acceleration vectors (m/s^2) to calculate roll and pitch, while the system outputs telemetry—such as survival time, score, and calibration instructions—to the LCD screen (Address 0x20). The stepper motor drive circuit also utilizes the I2C bus to receive angular movement commands, while the NeoPixel ring relies on board D5 to send data packets for LED color and brightness. If I wanted to attach other sensors to the device to expand the game (such as an I2C joystick or additional buttons), it would be through the STEMMA QT daisy-chain using the SDA, SCL, 3.3V, and GND pins, as it will allow the ESP32-S3 to seamlessly read the new I2C device connection.

I have attached a GitHub repository with ready-to-download software: [GitHub](#)

8.3 Operation

The DAMNED operates as a standalone, offline physical arcade console in this application. Because it is fully self-contained, it intentionally bypasses WiFi and ThingSpeak IoT features to maximize the processing power available for the 100Hz motor control loop and the Cartesian game engine.

- **Initialization and Calibration:** Upon receiving power, the ESP32-S3 begins its boot sequence. The stepper motor will rotate to find the Hall sensor magnet, establishing its 90-degree mechanical zero. Simultaneously, the LCD will display *"Calibrating... Keep Still!"*. For exactly 2 seconds, the device must remain untouched on a flat surface. The IMU takes 50 gravitational samples during this window to calculate a `calibration_offset`. This allows the game to function perfectly even if the table it is resting on is slightly slanted.
- **Normal Operation (Gameplay):** Once calibration clears, the LCDs the survival timer and score. The user physically holds the device and tilts it left or right. Tilting past the 10-degree deadzone commands the stepper motor arm to rotate proportionally, acting as the submarine's heading. On the NeoPixel ring, incoming Yellow LEDs represent Coins, and incoming Blue LEDs represent Icebergs. The player must steer the physical arm toward the yellow lights and away from the blue lights.

- **Game Over and Restarting:** If the virtual X/Y coordinates of an Iceberg intersect with the motor arm's angle at distance zero, the NeoPixel ring flashes solid red and the LCDs a crash warning. The system does not require a hard manual restart; the `submarine_engine.py` script automatically resets the score, clears the radar, and restarts the gameplay loop after 3 seconds.
- **Tuning and Adjustments:** If the user wants to play at a different resting angle (e.g., sitting up vs. lying down), they simply press the physical RESET button on the DAMNED board to trigger the 2-second IMU calibration sequence again.
 - *Adjusting Hardware Alignment:* If the NeoPixel lights do not visually line up with the tip of the physical motor arm, the user can modify the software without moving the wires. By opening `submarine_engine.py` and modifying the `self.LED_OFFSET` variable (e.g., 90, -90, or 180), the virtual radar will rotate to match the physical hardware configuration.
 - *Adjusting Difficulty:* The steering sensitivity can be modified in `code.py` by changing the `MAX_TURN_SPEED` (default is 1.6) or the `MIN_TILT_ANGLE` deadzone (default is 10.0)

Here is the flowchart for my project:

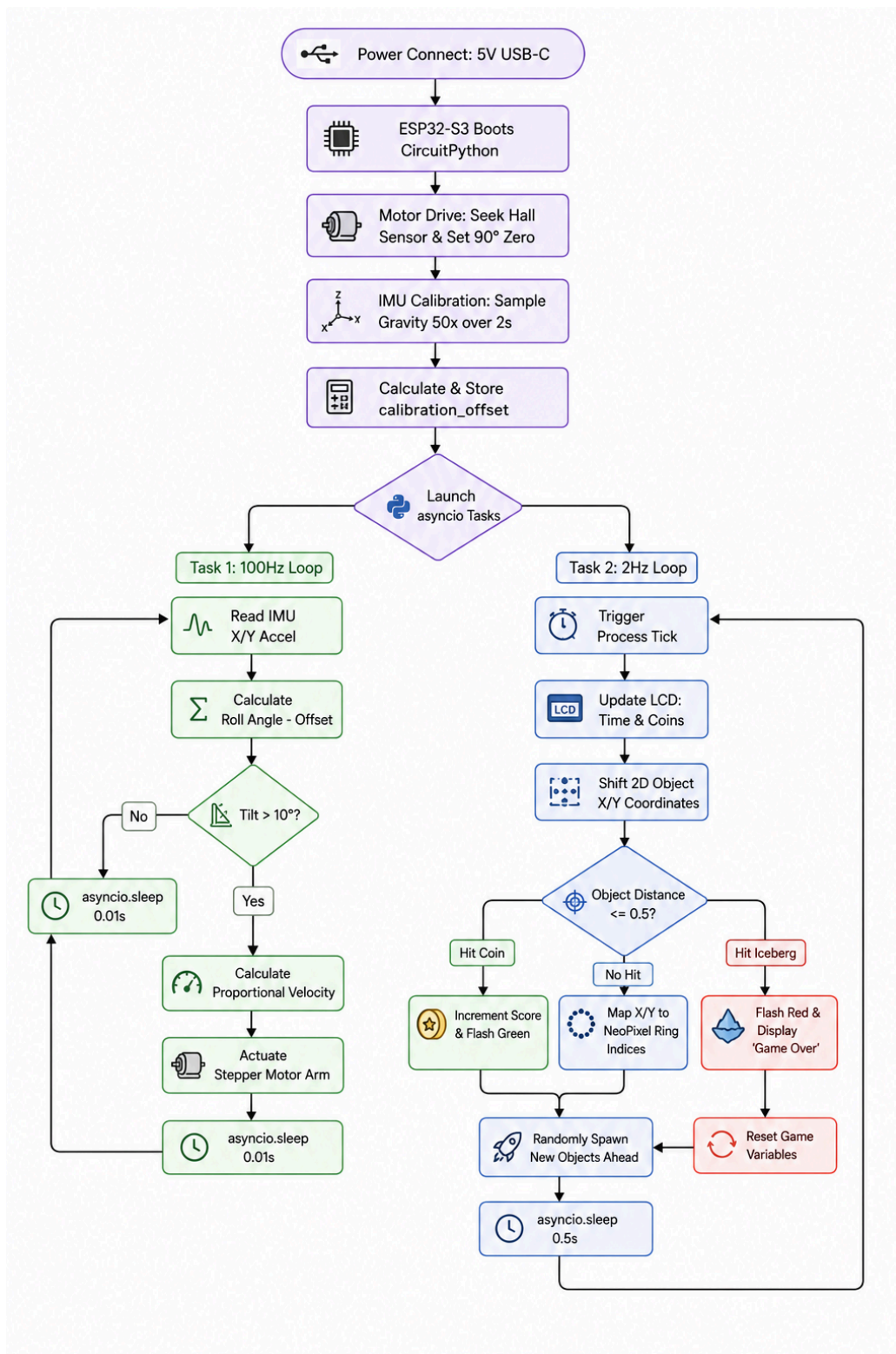
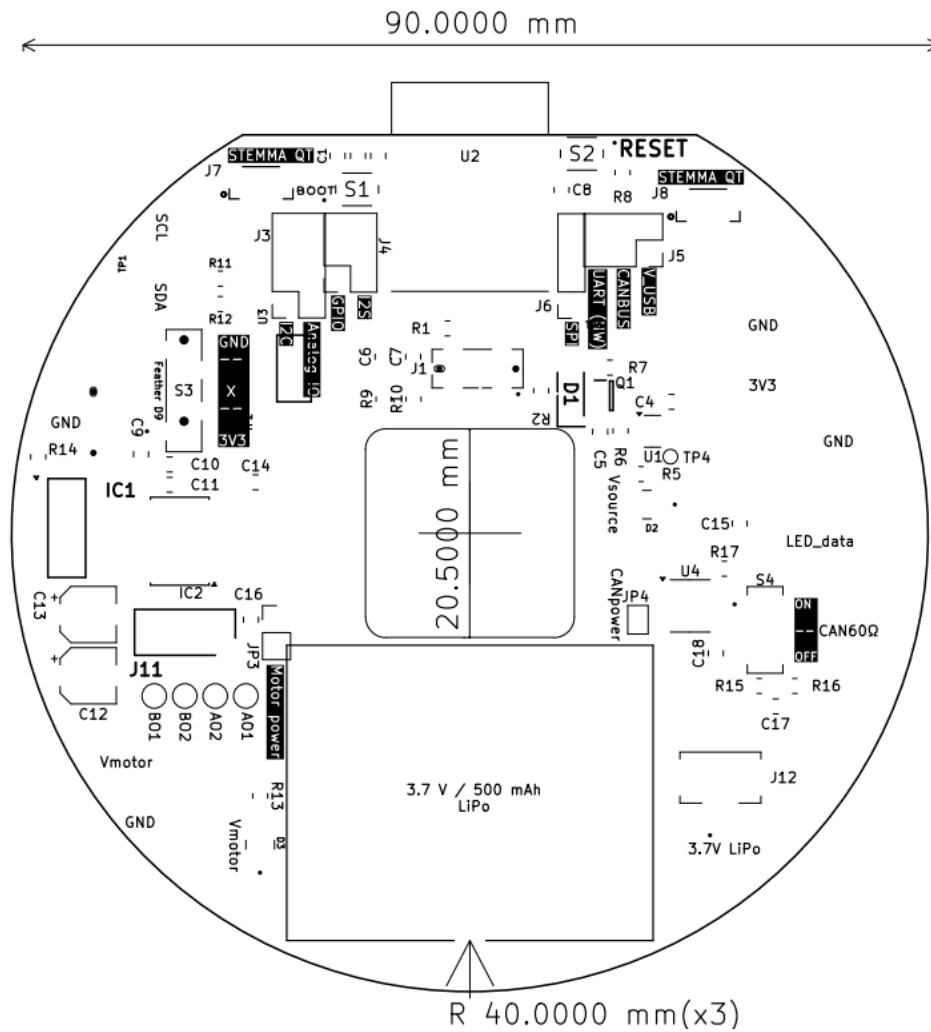


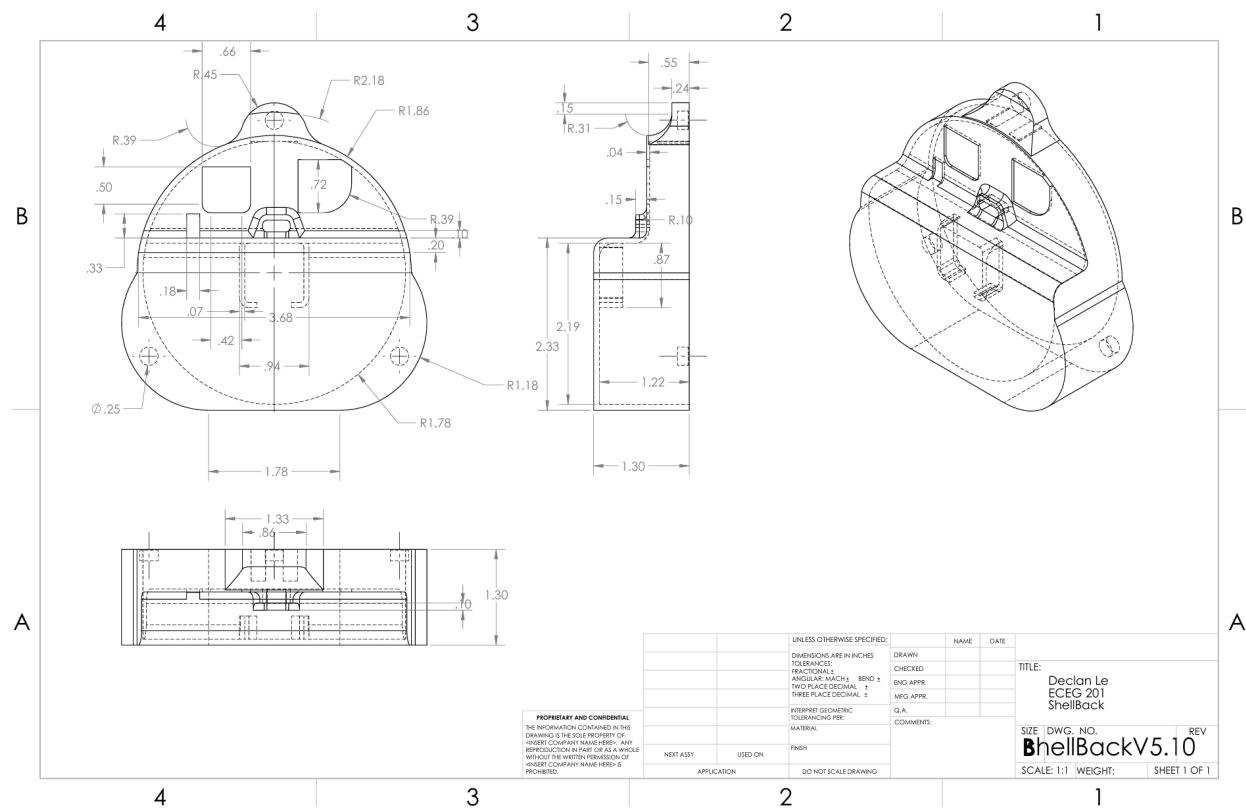
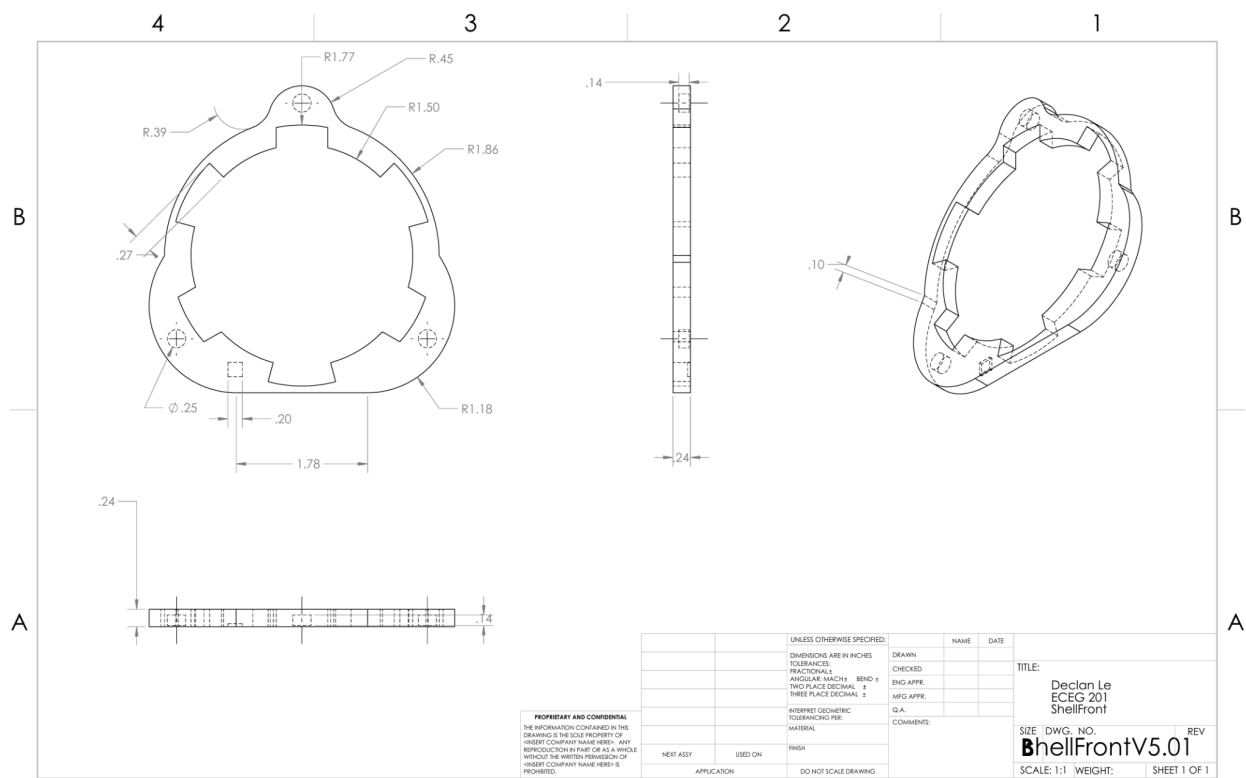
Figure 8: Flowchart of the Submarine game project

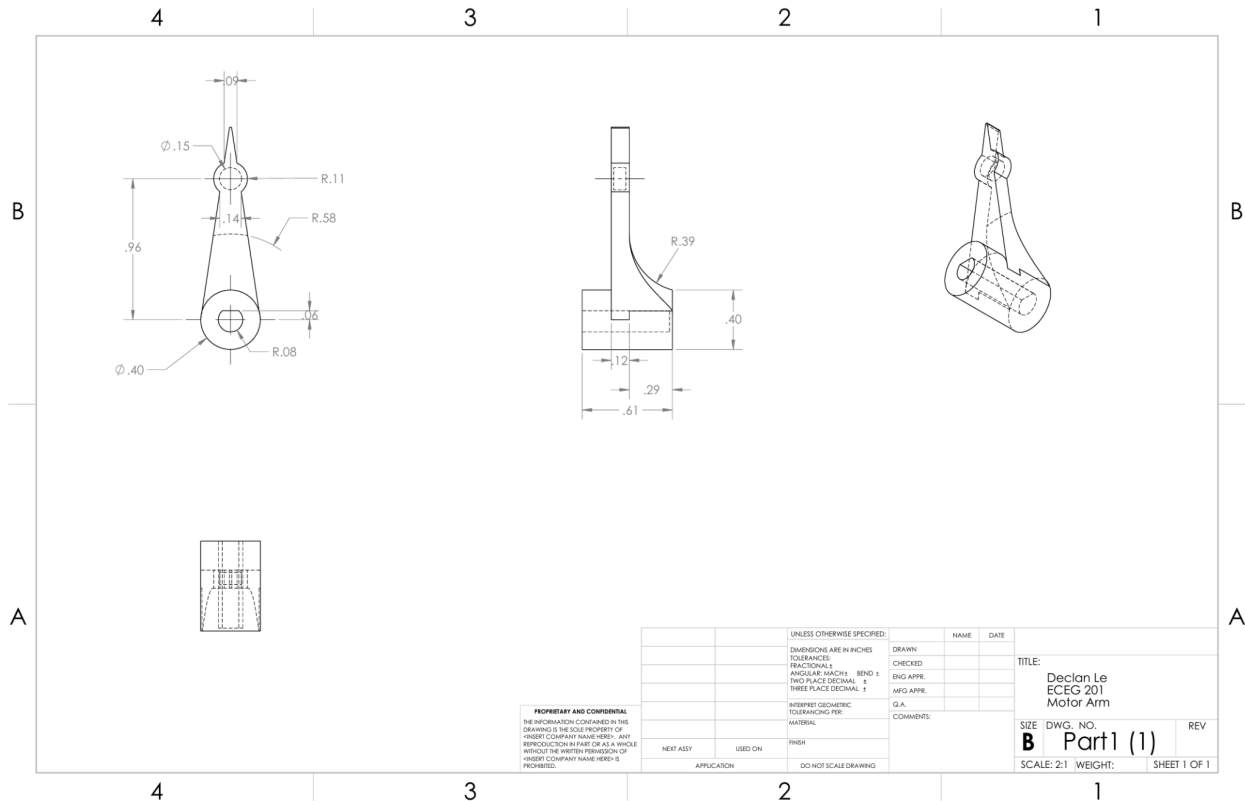
9. Mechanical Layout and Dimensions

9.1 Printed Circuit Board



9.2 Case





10 Citations

- [1] Espressif Systems, "ESP32-S3 Series Datasheet," Espressif Systems, 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32-s3_datasheet_en.pdf.
- [2] MathWorks, "Stepper Motor Basics," MathWorks Documentation, 2023. [Online]. Available: <https://www.mathworks.com/help/phymod/sps/ug/stepper-motor.html>.
- [3] Adafruit Industries, "Adafruit TDK InvenSense ICM-20948 9-DoF IMU," Product ID 4554. [Online]. Available: <https://www.adafruit.com/product/4554>.
- [4] Adafruit Industries, "I2C / SPI character LCD backpack," Product ID 292. [Online]. Available: <https://www.adafruit.com/product/292>.
- [5] Python Software Foundation, "asyncio — Asynchronous I/O," Python Documentation. [Online]. Available: <https://docs.python.org/3/library/asyncio.html>.